

Install libraries

```
!pip install pandas numpy matplotlib seaborn plotly scikit-learn wordcloud -q
```

Import libraries

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
from wordcloud import WordCloud
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.linear_model import LinearRegression
import warnings
warnings.filterwarnings("ignore")
```

Load Dataset

```
df = pd.read_csv("/content/ai_jobs_market_2025_2026.csv")
```

```
df.head()
```

	job_id	job_title	job_category	experience_level	years_of_experience	education_required	annual_salary_usd	salary_min_usd	sa
0	AIJOB0001	AI Agent Developer	AI Engineering	Senior (6-9 yrs)	7	Master's	239000.0	155000	
1	AIJOB0002	Prompt Engineer	AI Engineering	Senior (6-9 yrs)	2	Bachelor's	166000.0	90000	
2	AIJOB0003	LLM Engineer	AI Engineering	Senior (6-9 yrs)	4	Associate's	360000.0	160000	
3	AIJOB0004	Data Engineer (AI)	Data Engineering	Senior (6-9 yrs)	3	Bachelor's	161000.0	130000	
4	AIJOB0005	AI Product Manager	Product	Lead (10+ yrs)	5	Bootcamp/Self-taught	283000.0	140000	

5 rows × 25 columns

Understand Dataset

```
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
```

Rows: 1500
Columns: 25

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1500 entries, 0 to 1499
Data columns (total 25 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   job_id                                1500 non-null   object
1   job_title                             1500 non-null   object
2   job_category                          1500 non-null   object
3   experience_level                       1500 non-null   object
4   years_of_experience                    1500 non-null   int64
5   education_required                    1500 non-null   object
6   annual_salary_usd                     1500 non-null   float64
7   salary_min_usd                        1500 non-null   int64
8   salary_max_usd                        1500 non-null   int64
9   city                                  1500 non-null   object
10  country                               1500 non-null   object
11  remote_work                           1500 non-null   object
12  company_size                           1500 non-null   object
13  industry                              1500 non-null   object
14  required_skills                       1500 non-null   object
15  ai_salary_premium_pct                 1500 non-null   float64
```

```
16 demand_score 1500 non-null int64
17 demand_growth_yoy_pct 1500 non-null float64
18 benefits_score_10 1500 non-null float64
19 posting_year 1500 non-null int64
20 posting_month 1500 non-null int64
21 is_senior 1500 non-null int64
22 is_remote_friendly 1500 non-null int64
23 is_llm_role 1500 non-null int64
24 salary_tier 1500 non-null object
dtypes: float64(4), int64(9), object(12)
memory usage: 293.1+ KB
```

df.describe(include="all").T

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
job_id	1500	1500	AIJOB1500	1	NaN	NaN	NaN	NaN	NaN	NaN	NaN
job_title	1500	25	LLM Engineer	75	NaN	NaN	NaN	NaN	NaN	NaN	NaN
job_category	1500	12	AI Engineering	736	NaN	NaN	NaN	NaN	NaN	NaN	NaN
experience_level	1500	4	Entry (0-2 yrs)	385	NaN	NaN	NaN	NaN	NaN	NaN	NaN
years_of_experience	1500.0	NaN	NaN	NaN	6.216	2.675216	1.0	4.0	6.0	8.0	15.0
education_required	1500	5	Master's	316	NaN	NaN	NaN	NaN	NaN	NaN	NaN
annual_salary_usd	1500.0	NaN	NaN	NaN	194892.0	66506.822013	90000.0	144750.0	180000.0	236250.0	384000.0
salary_min_usd	1500.0	NaN	NaN	NaN	135448.666667	24448.950878	90000.0	120000.0	140000.0	155000.0	180000.0
salary_max_usd	1500.0	NaN	NaN	NaN	257537.333333	39852.822207	180000.0	218000.0	270000.0	290000.0	320000.0
city	1500	20	San Francisco	92	NaN	NaN	NaN	NaN	NaN	NaN	NaN
country	1500	14	USA	515	NaN	NaN	NaN	NaN	NaN	NaN	NaN
remote_work	1500	3	Hybrid	686	NaN	NaN	NaN	NaN	NaN	NaN	NaN
company_size	1500	5	Mid-size (501-5000)	312	NaN	NaN	NaN	NaN	NaN	NaN	NaN
industry	1500	12	Automotive	138	NaN	NaN	NaN	NaN	NaN	NaN	NaN
required_skills	1500	1500	Cybersecurity ML Security Red Teaming Python R...	1	NaN	NaN	NaN	NaN	NaN	NaN	NaN
ai_salary_premium_pct	1500.0	NaN	NaN	NaN	10.8582	4.029742	3.0	8.2	10.5	14.2	18.0
demand_score	1500.0	NaN	NaN	NaN	87.523333	8.026315	68.0	82.0	89.0	95.0	98.0
demand_growth_yoy_pct	1500.0	NaN	NaN	NaN	31.116333	22.046343	5.0	15.375	23.4	42.7	87.0
benefits_score_10	1500.0	NaN	NaN	NaN	7.897333	1.102846	6.0	6.9	7.9	8.9	9.0
posting_year	1500.0	NaN	NaN	NaN	2025.584	0.493058	2025.0	2025.0	2026.0	2026.0	2026.0
posting_month	1500.0	NaN	NaN	NaN	3.968	3.270388	1.0	2.0	3.0	5.0	12.0
is_senior	1500.0	NaN	NaN	NaN	0.496667	0.500156	0.0	0.0	0.0	1.0	1.0
is_remote_friendly	1500.0	NaN	NaN	NaN	0.754	0.430822	0.0	1.0	1.0	1.0	1.0
is_llm_role	1500.0	NaN	NaN	NaN	0.218	0.413025	0.0	0.0	0.0	0.0	1.0
salary_tier	1500	5	Senior (\$200-300k)	467	NaN	NaN	NaN	NaN	NaN	NaN	NaN

Column names

df.columns.tolist()

['job_id',
 'job_title',
 'job_category',
 'experience_level',
 'years_of_experience',
 'education_required',
 'annual_salary_usd',
 'salary_min_usd',
 'salary_max_usd',
 'city',
 'country',
 'remote_work',
 'company_size',
 'industry',
 'required_skills',
 'ai_salary_premium_pct',
 'demand_score',
 'demand_growth_yoy_pct',
 'benefits_score_10',
 'posting_year',
 'posting_month',
 'is_senior',

```
'is_remote_friendly',  
'is_llm_role',  
'salary_tier']
```

Data Cleaning Missing values

```
missing = df.isnull().sum().sort_values(ascending=False)  
missing
```

	0
job_id	0
job_title	0
job_category	0
experience_level	0
years_of_experience	0
education_required	0
annual_salary_usd	0
salary_min_usd	0
salary_max_usd	0
city	0
country	0
remote_work	0
company_size	0
industry	0
required_skills	0
ai_salary_premium_pct	0
demand_score	0
demand_growth_yoy_pct	0
benefits_score_10	0
posting_year	0
posting_month	0
is_senior	0
is_remote_friendly	0
is_llm_role	0
salary_tier	0

dtype: int64

Missing_percentage

```
missing_percentage = (  
    df.isnull().sum() / len(df) * 100  
)  
.sort_values(ascending=False)  
  
missing_percentage
```

	0
job_id	0.0
job_title	0.0
job_category	0.0
experience_level	0.0
years_of_experience	0.0
education_required	0.0
annual_salary_usd	0.0
salary_min_usd	0.0
salary_max_usd	0.0
city	0.0
country	0.0
remote_work	0.0
company_size	0.0
industry	0.0
required_skills	0.0
ai_salary_premium_pct	0.0
demand_score	0.0
demand_growth_yoy_pct	0.0
benefits_score_10	0.0
posting_year	0.0
posting_month	0.0
is_senior	0.0
is_remote_friendly	0.0
is_ilm_role	0.0
salary_tier	0.0

dtype: float64

Remove duplicate records

```
df = df.drop_duplicates()

print("Dataset shape:", df.shape)
```

Dataset shape: (1500, 25)

Check salary

```
df["annual_salary_usd"].describe()
```

	annual_salary_usd
count	1500.000000
mean	194892.000000
std	66506.822013
min	90000.000000
25%	144750.000000
50%	180000.000000
75%	236250.000000
max	384000.000000

dtype: float64

Remove invalid salary values

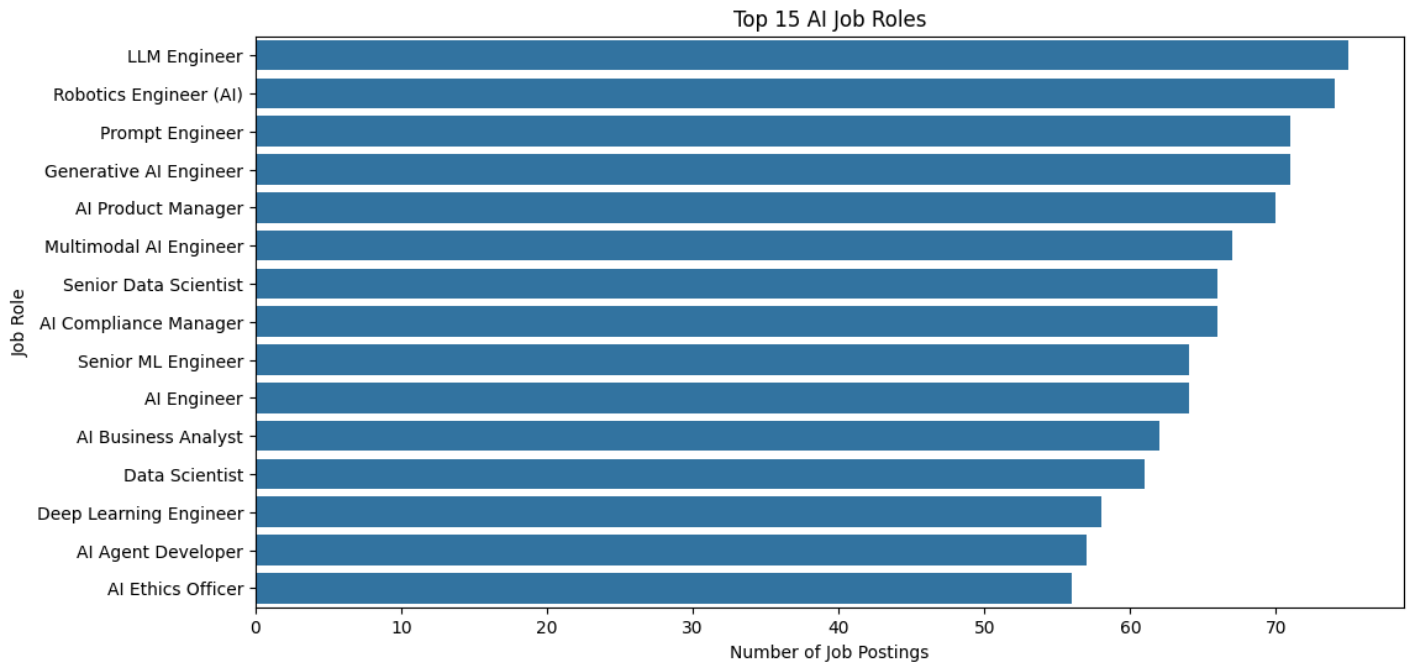
```
df = df[
    (df["annual_salary_usd"] > 0) &
    (df["annual_salary_usd"] < 1000000)
]
```

```
print(df.shape)
```

```
(1500, 25)
```

Basic EDA Top Job Roles

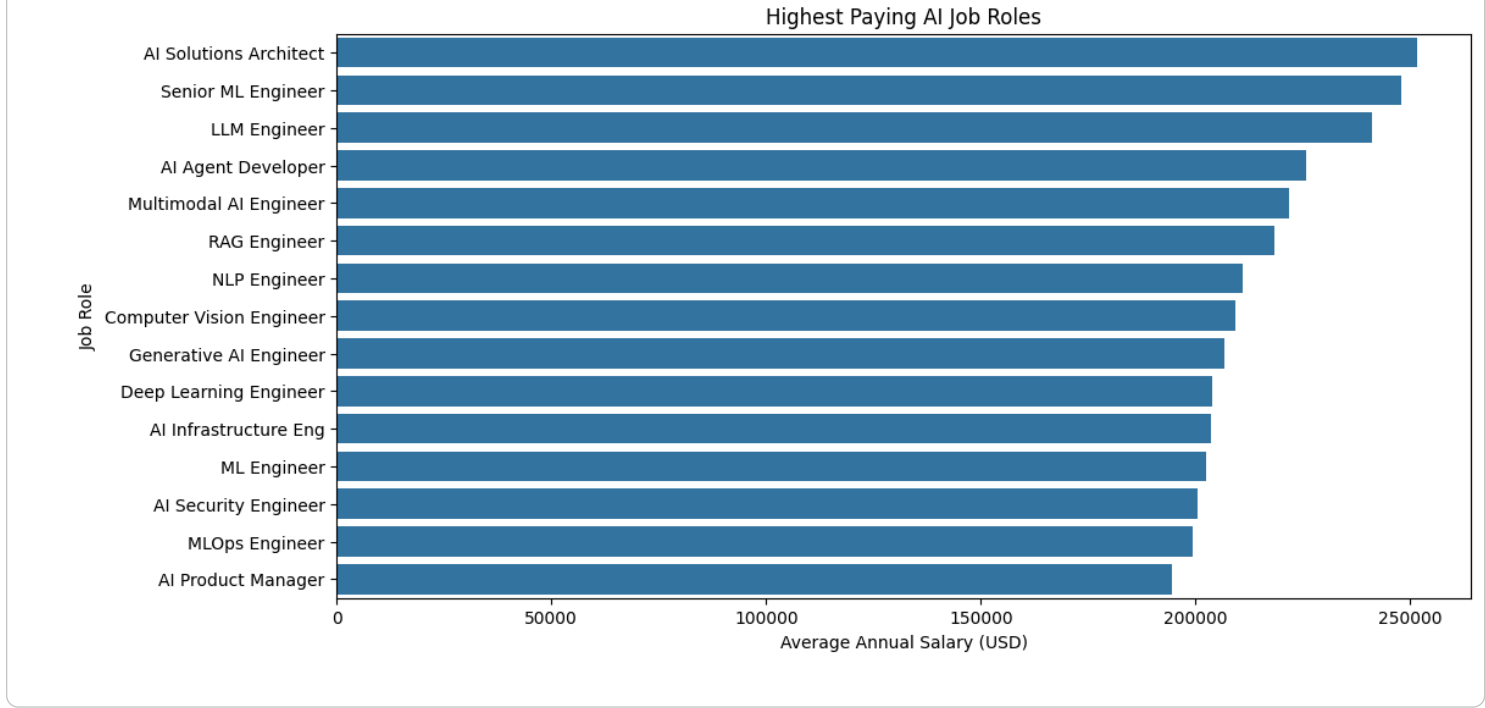
```
top_roles = df["job_title"].value_counts().head(15)
plt.figure(figsize=(12,6))
sns.barplot(
    x=top_roles.values,
    y=top_roles.index
)
plt.title("Top 15 AI Job Roles")
plt.xlabel("Number of Job Postings")
plt.ylabel("Job Role")
plt.show()
```



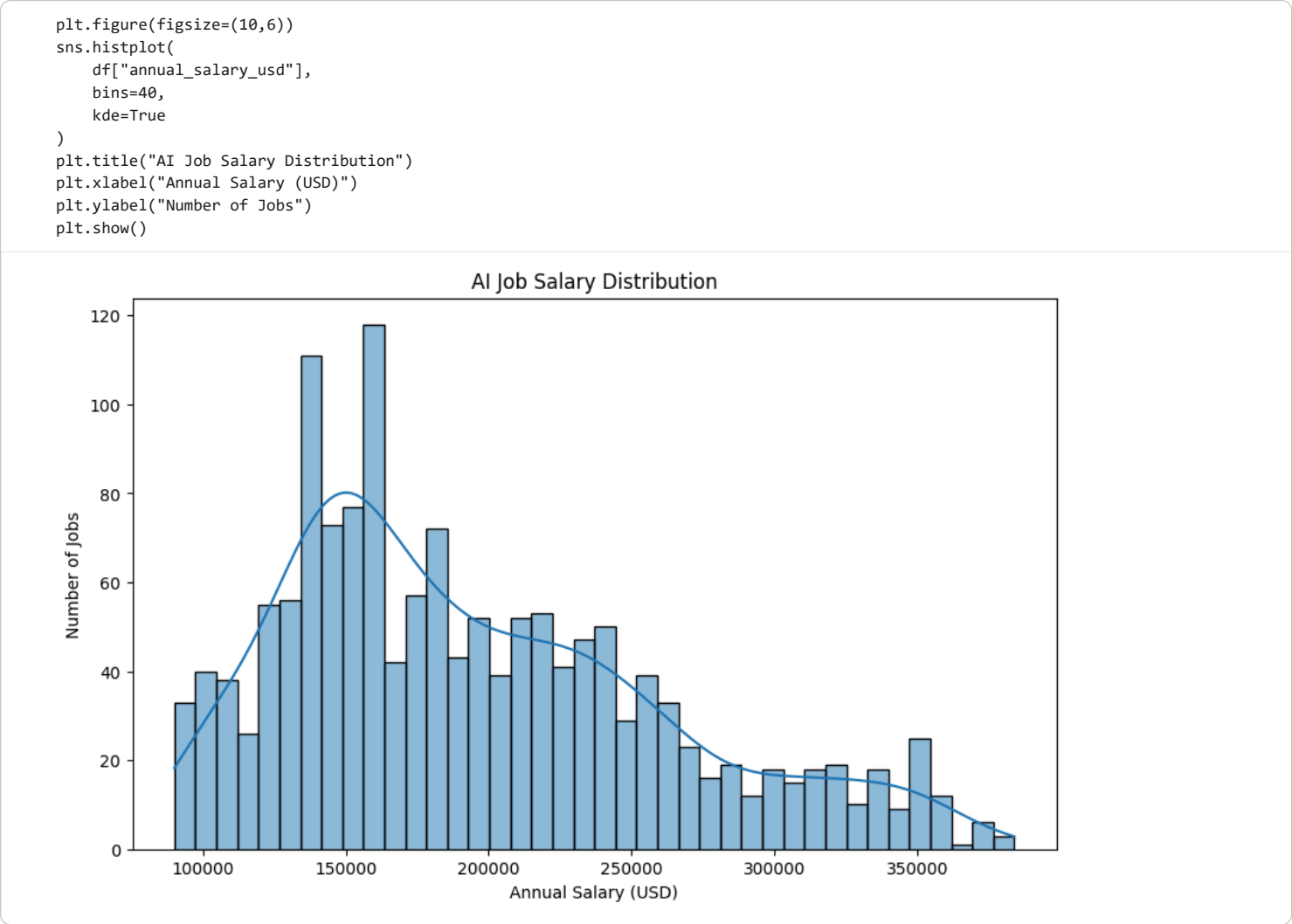
Average Salary by Job Role

```
salary_role = (
    df.groupby("job_title")["annual_salary_usd"]
    .mean()
    .sort_values(ascending=False)
    .head(15)
)
plt.figure(figsize=(12,6))

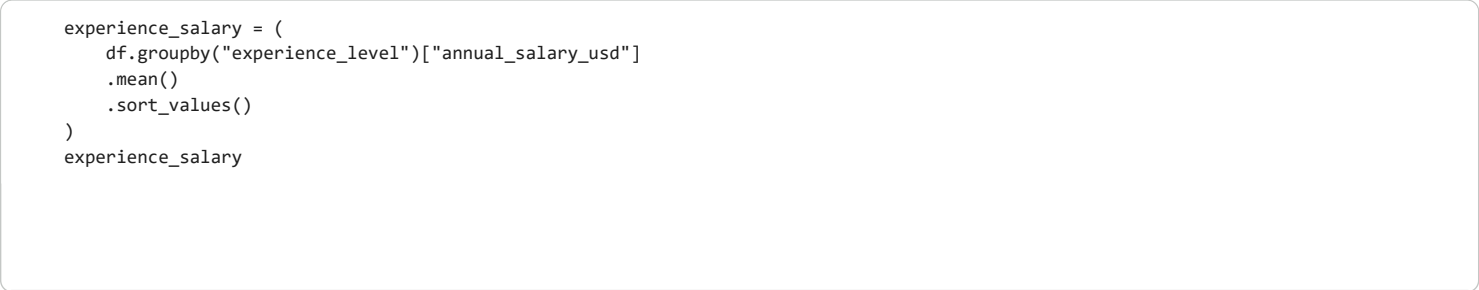
sns.barplot(
    x=salary_role.values,
    y=salary_role.index
)
plt.title("Highest Paying AI Job Roles")
plt.xlabel("Average Annual Salary (USD)")
plt.ylabel("Job Role")
plt.show()
```



Salary Distribution



Experience vs Salary



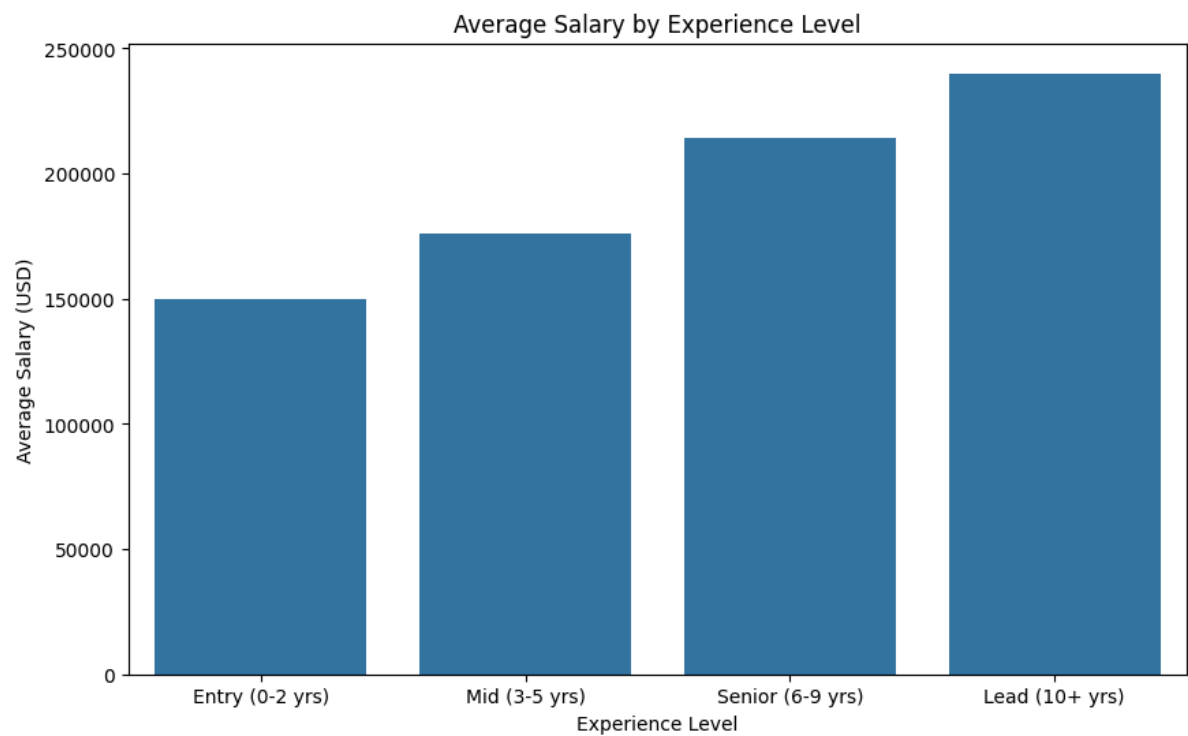
annual_salary_usd	
experience_level	
Entry (0-2 yrs)	150038.961039
Mid (3-5 yrs)	175983.783784
Senior (6-9 yrs)	214280.219780
Lead (10+ yrs)	240055.118110

dtype: float64

Visualization

```
plt.figure(figsize=(10,6))
sns.barplot(
    x=experience_salary.index,
    y=experience_salary.values
)

plt.title("Average Salary by Experience Level")
plt.xlabel("Experience Level")
plt.ylabel("Average Salary (USD)")
plt.show()
```



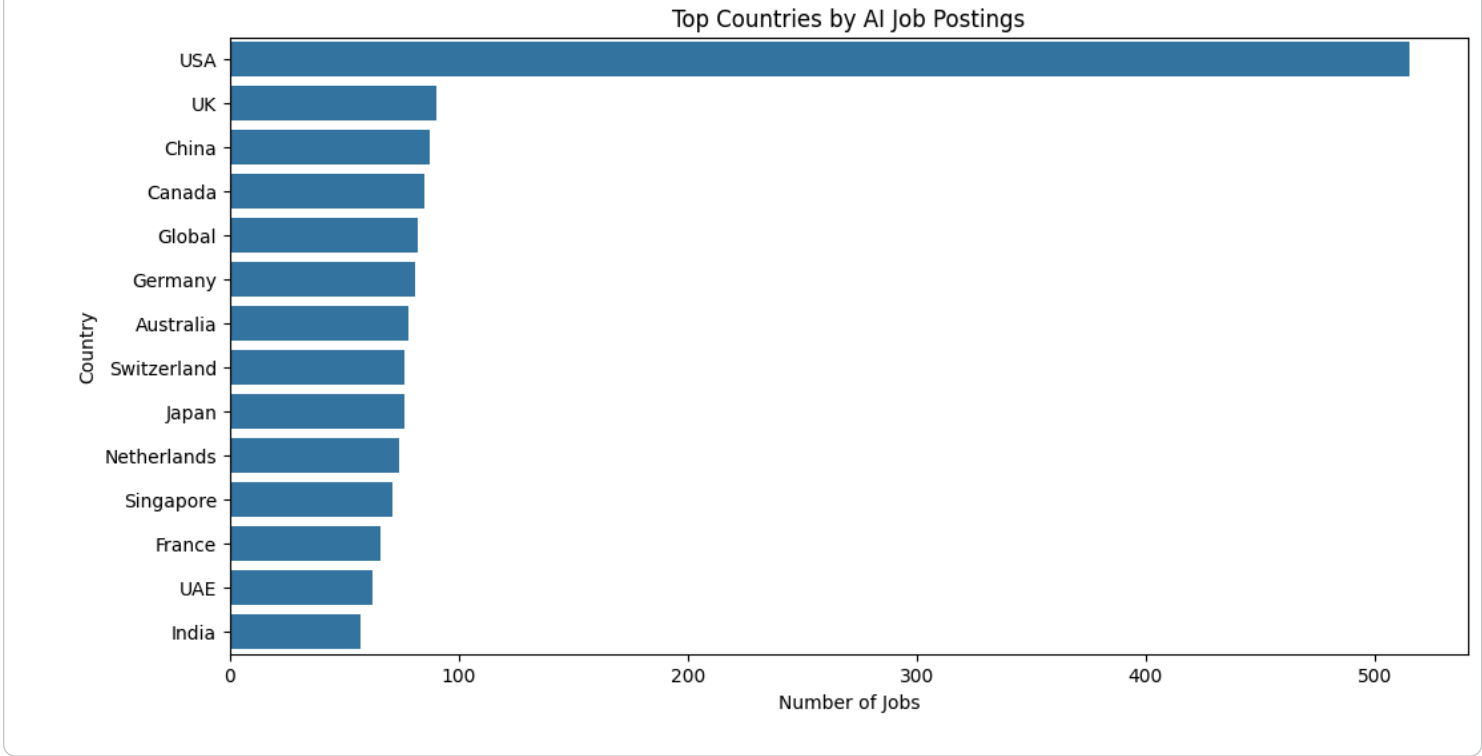
Country Analysis Job demand by country

```
country_jobs = df["country"].value_counts().head(15)
plt.figure(figsize=(12,6))

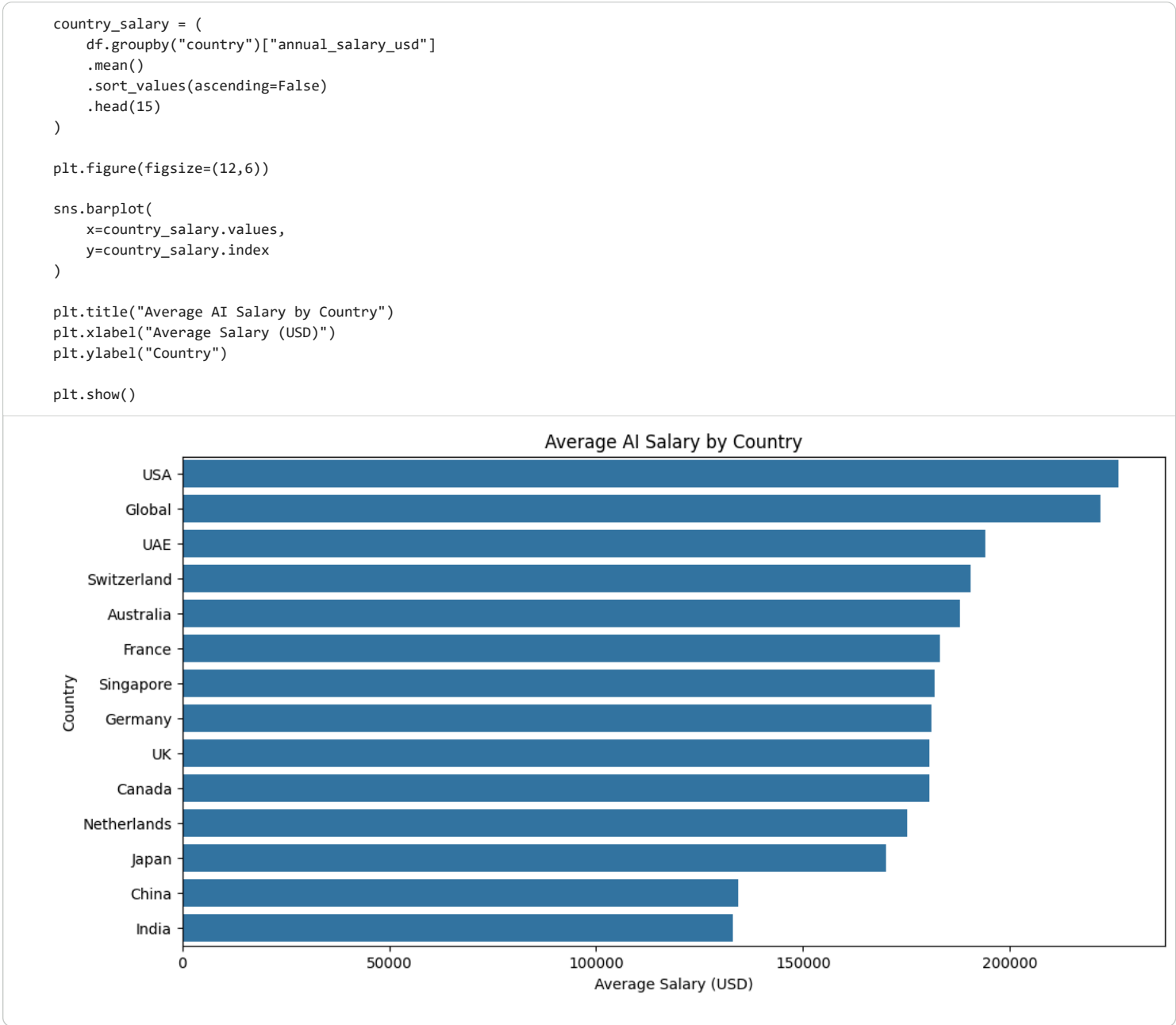
sns.barplot(
    x=country_jobs.values,
    y=country_jobs.index
)

plt.title("Top Countries by AI Job Postings")
plt.xlabel("Number of Jobs")
plt.ylabel("Country")

plt.show()
```



Salary by Country



Remote Work Analysis


```
remote_counts = df["remote_work"].value_counts()
```

```
remote_counts
```

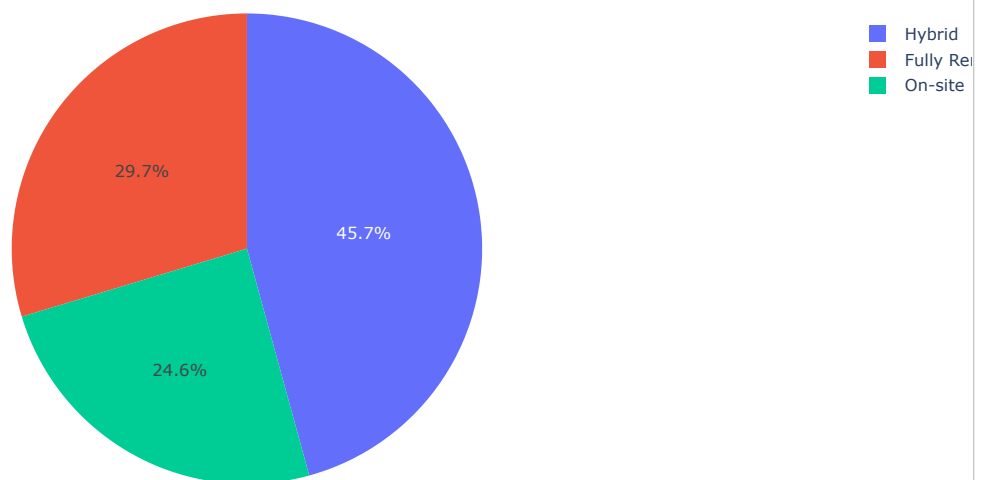
count	
remote_work	
Hybrid	686
Fully Remote	445
On-site	369

```
dtype: int64
```

```
fig = px.pie(
    values=remote_counts.values,
    names=remote_counts.index,
    title="Remote Work Distribution"
)

fig.show()
```

Remote Work Distribution



Remote vs Salary

```
remote_salary = (
    df.groupby("remote_work")["annual_salary_usd"]
    .mean()
    .sort_values(ascending=False)
)

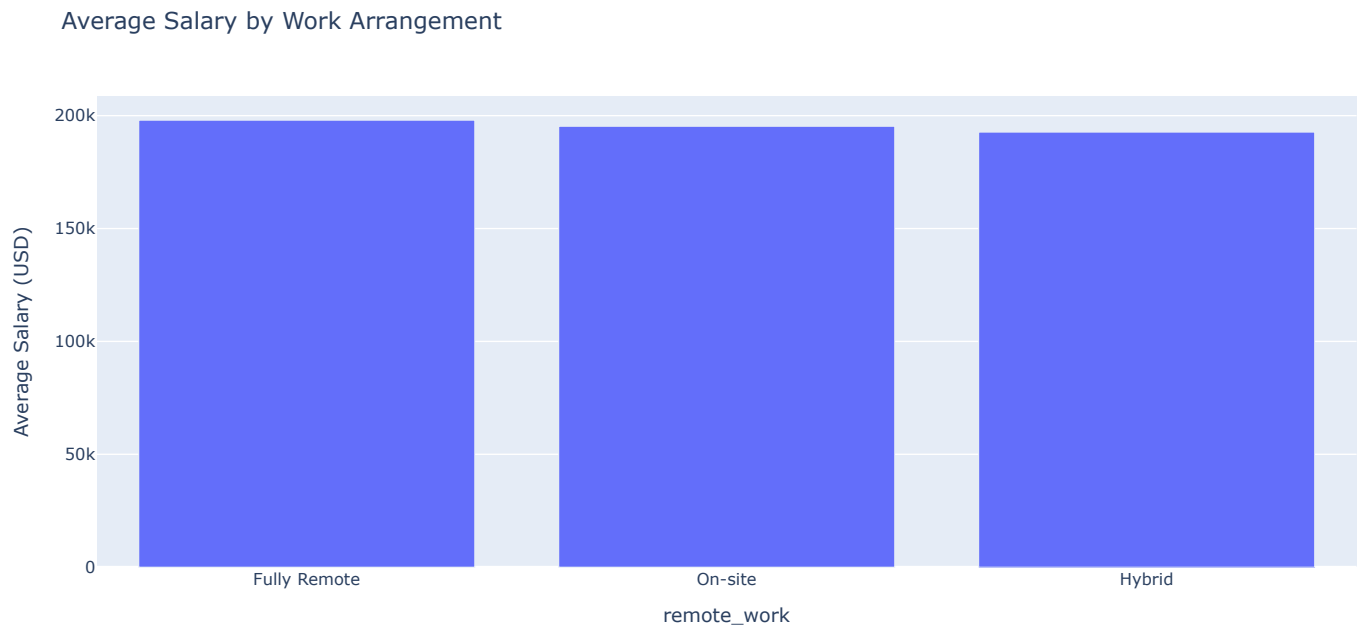
remote_salary
```

annual_salary_usd	
remote_work	
Fully Remote	197967.191011
On-site	195268.834688
Hybrid	192694.460641

```
dtype: float64
```

```
fig = px.bar(
    remote_salary,
    x=remote_salary.index,
    y=remote_salary.values,
    title="Average Salary by Work Arrangement",
    labels={
        "x": "Work Arrangement",
        "y": "Average Salary (USD)"
    }
)
```

```
fig.show()
```



Company Size Analysis

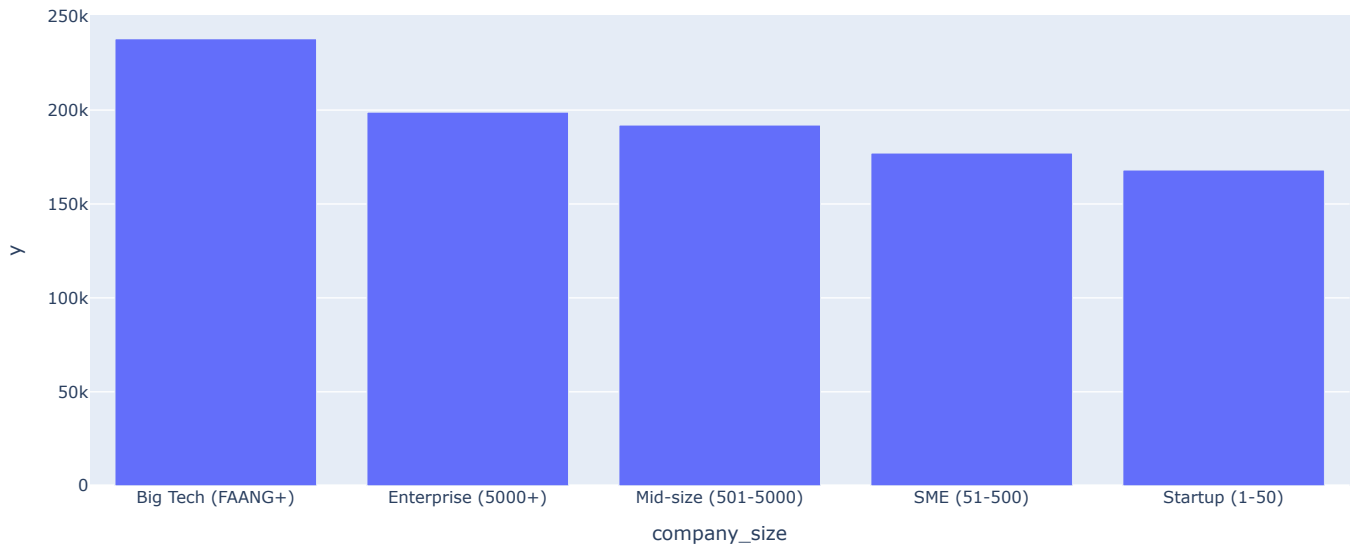
```
company_salary = (  
    df.groupby("company_size")["annual_salary_usd"]  
        .mean()  
        .sort_values(ascending=False)  
)  
  
company_salary
```

	annual_salary_usd
company_size	
Big Tech (FAANG+)	237951.170569
Enterprise (5000+)	198876.094276
Mid-size (501-5000)	192016.666667
SME (51-500)	177118.666667
Startup (1-50)	168080.821918

dtype: float64

```
fig = px.bar(  
    company_salary,  
    x=company_salary.index,  
    y=company_salary.values,  
    title="Average Salary by Company Size"  
)  
  
fig.show()
```

Average Salary by Company Size

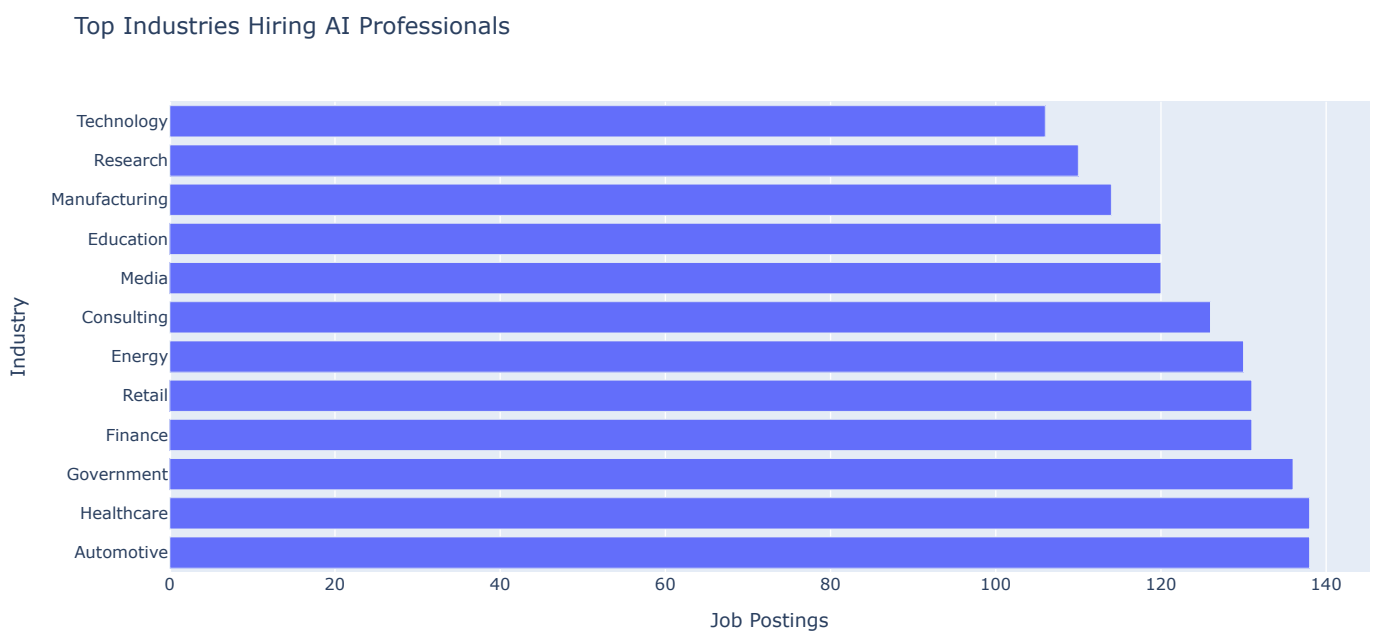


Industry Analysis

```
industry_jobs = df["industry"].value_counts().head(15)

fig = px.bar(
    x=industry_jobs.values,
    y=industry_jobs.index,
    orientation="h",
    title="Top Industries Hiring AI Professionals",
    labels={
        "x": "Job Postings",
        "y": "Industry"
    }
)

fig.show()
```



Demand score

```
role_demand = (
    df.groupby("job_title")["demand_score"]
    .mean()
    .sort_values(ascending=False)
    .head(15)
)
```

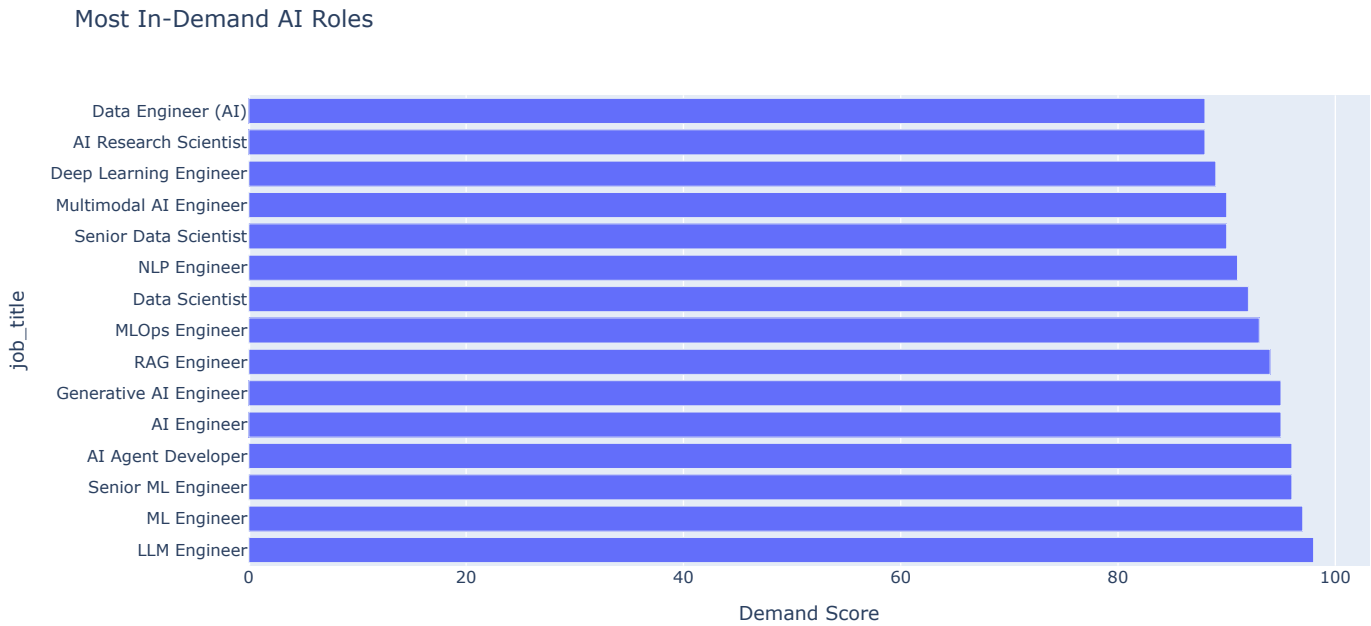
role_demand

demand_score	
job_title	
LLM Engineer	98.0
ML Engineer	97.0
Senior ML Engineer	96.0
AI Agent Developer	96.0
AI Engineer	95.0
Generative AI Engineer	95.0
RAG Engineer	94.0
MLOps Engineer	93.0
Data Scientist	92.0
NLP Engineer	91.0
Senior Data Scientist	90.0
Multimodal AI Engineer	90.0
Deep Learning Engineer	89.0
AI Research Scientist	88.0
Data Engineer (AI)	88.0

dtype: float64

```
fig = px.bar(
    role_demand,
    x=role_demand.values,
    y=role_demand.index,
    orientation="h",
    title="Most In-Demand AI Roles",
    labels={
        "x": "Demand Score",
        "y": "Job Role"
    }
)

fig.show()
```



Demand Growth

```
growth = (
    df.groupby("job_title")["demand_growth_yoy_pct"]
    .mean()
    .sort_values(ascending=False)
    .head(15)
)
```

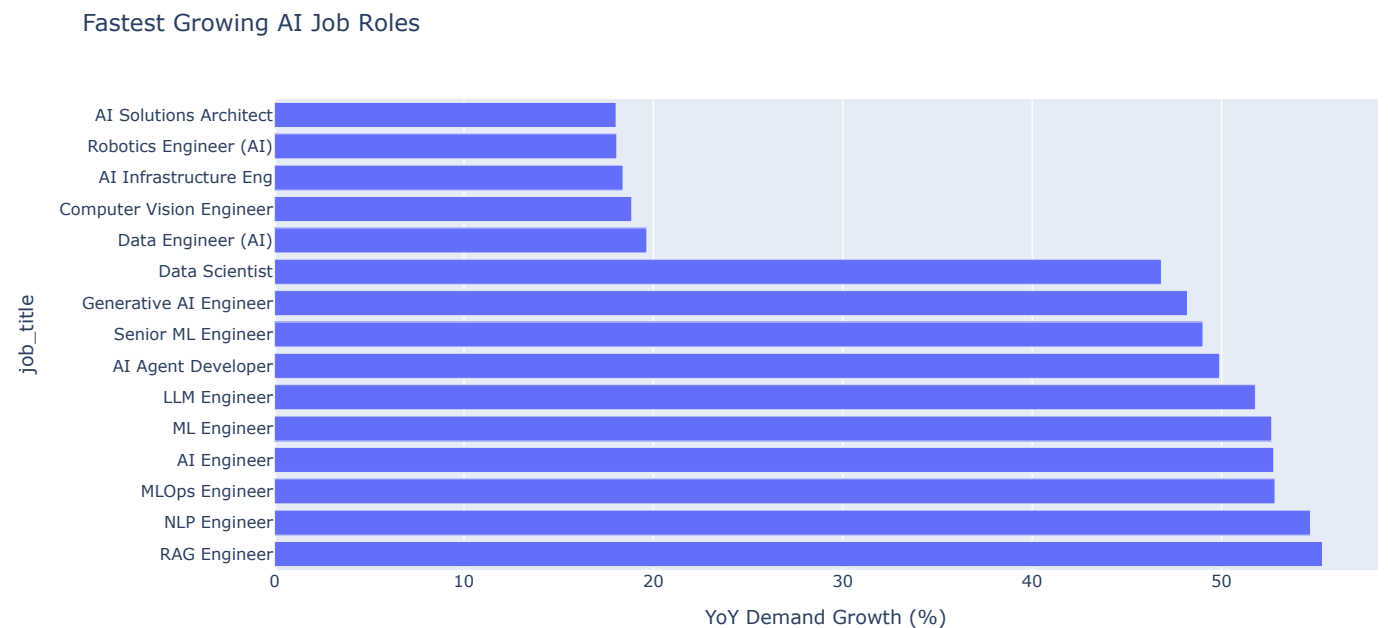
growth

job_title	demand_growth_yoy_pct
RAG Engineer	55.324528
NLP Engineer	54.694545
MLOps Engineer	52.819608
AI Engineer	52.750000
ML Engineer	52.645455
LLM Engineer	51.784000
AI Agent Developer	49.896491
Senior ML Engineer	49.017187
Generative AI Engineer	48.198592
Data Scientist	46.821311
Data Engineer (AI)	19.652941
Computer Vision Engineer	18.856522
AI Infrastructure Eng	18.398182
Robotics Engineer (AI)	18.067568
AI Solutions Architect	18.032692

dtype: float64

```
fig = px.bar(
    growth,
    x=growth.values,
    y=growth.index,
    orientation="h",
    title="Fastest Growing AI Job Roles",
    labels={
        "x": "YoY Demand Growth (%)",
        "y": "Job Role"
    }
)

fig.show()
```



Extract skills

```
skills_series = (
    df["required_skills"]
    .dropna()
    .astype(str)
    .str.split("|")
    .explode()
```

```

        .str.strip()
    )

    skill_counts = skills_series.value_counts()

    skill_counts.head(20)

```

required_skills	count
Python	942
SQL	452
Cloud	429
Leadership	380
Communication	378
Research	376
Agile	351
Statistics	350
Linux	320
Problem Solving	314
PyTorch	302
Git	295
Fine-tuning	164
LLMs	125
Kubernetes	123
CUDA	117
Documentation	108
LLM APIs	103
Distributed Systems	99
APIs	98

dtype: int64

Most demanded skills

```

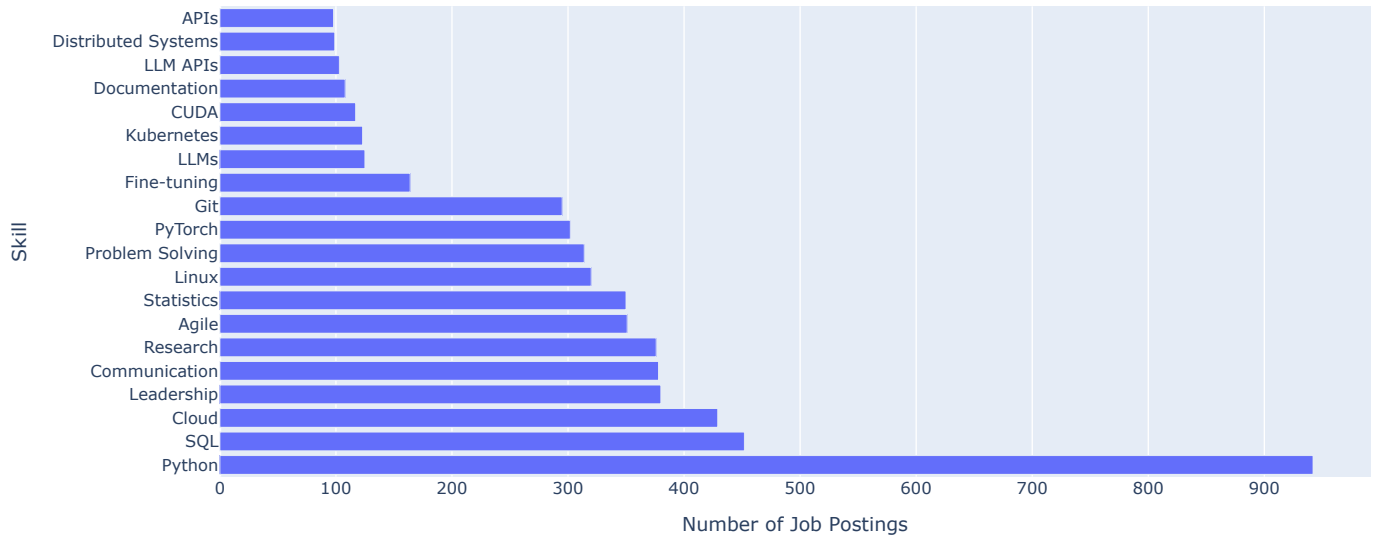
top_skills = skill_counts.head(20)

fig = px.bar(
    x=top_skills.values,
    y=top_skills.index,
    orientation="h",
    title="Top 20 Most In-Demand Skills",
    labels={
        "x": "Number of Job Postings",
        "y": "Skill"
    }
)

fig.show()

```

Top 20 Most In-Demand Skills



Skills WordCloud

```
skill_text = " ".join(skills_series.tolist())

wordcloud = WordCloud(
    width=1200,
    height=600,
    background_color="white"
).generate(skill_text)

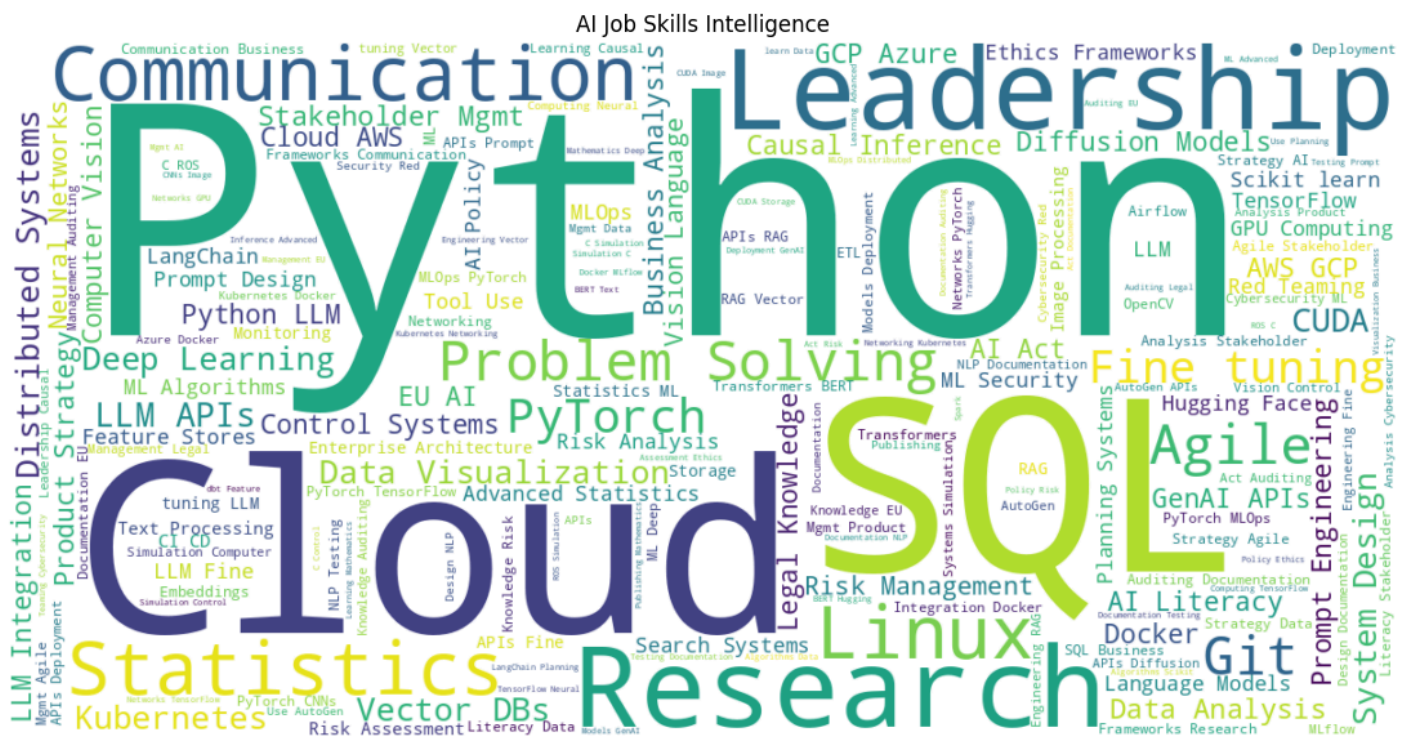
plt.figure(figsize=(15,7))

plt.imshow(wordcloud, interpolation="bilinear")

plt.axis("off")

plt.title("AI Job Skills Intelligence")

plt.show()
```



Skill vs Salary

```
skill_salary_data = []

for skill in skill_counts.head(20).index:

    mask = df["required_skills"].fillna("").str.contains(
        skill,
        case=False,
        regex=False
    )

    avg_salary = df.loc[
        mask,
        "annual_salary_usd"
    ].mean()

    skill_salary_data.append(
        [skill, avg_salary]
    )

skill_salary = pd.DataFrame(
    skill_salary_data,
    columns=["Skill", "Average Salary"]
)

skill_salary = skill_salary.sort_values(
    "Average Salary",
    ascending=False
)

skill_salary
```

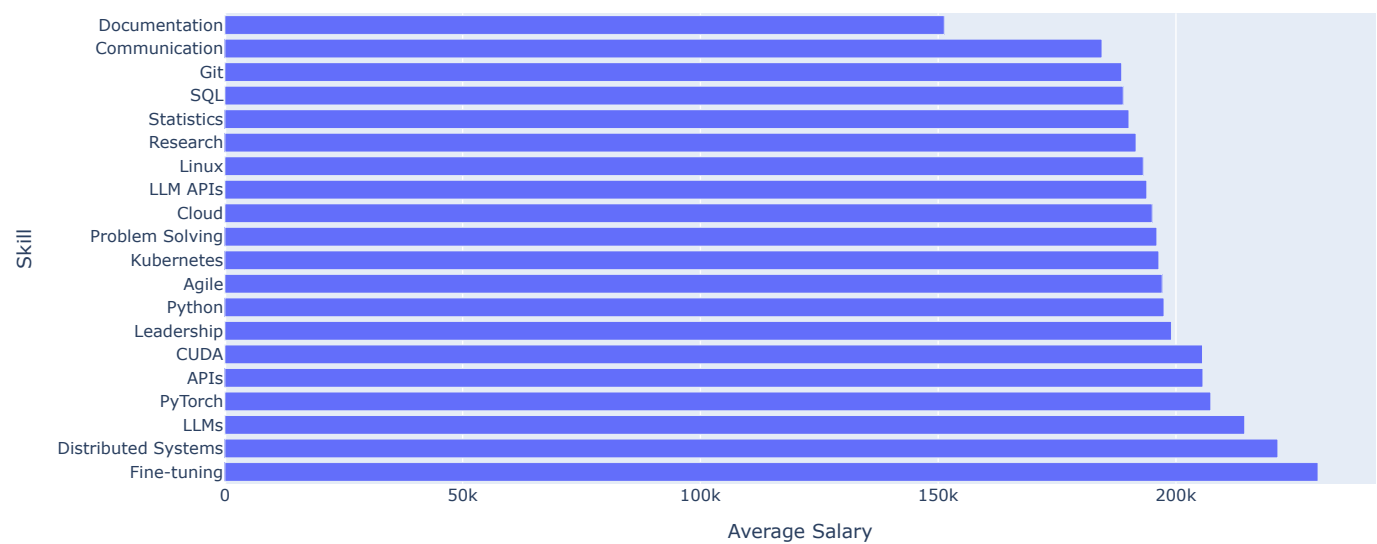
	Skill	Average Salary	
12	Fine-tuning	229891.262136	
18	Distributed Systems	221442.424242	
13	LLMs	214464.000000	
10	PyTorch	207335.099338	
19	APIs	205679.536680	
15	CUDA	205589.743590	
3	Leadership	199056.198347	
0	Python	197499.787686	
6	Agile	197110.588235	
14	Kubernetes	196414.634146	
9	Problem Solving	195992.356688	
2	Cloud	195015.315315	
17	LLM APIs	193873.786408	
8	Linux	193130.000000	
5	Research	191626.997245	
7	Statistics	190127.466667	
1	SQL	188920.853081	
11	Git	188591.864407	
4	Communication	184459.726027	
16	Documentation	151287.037037	

Next steps: [Generate code with skill_salary](#) [New interactive sheet](#)

```
fig = px.bar(
    skill_salary,
    x="Average Salary",
    y="Skill",
    orientation="h",
    title="Skills Associated With Higher Average Salaries"
)

fig.show()
```


Skills Associated With Higher Average Salaries



LLM / GenAI Jobs

```
llm_jobs = df["is_llm_role"].value_counts()

llm_jobs
```

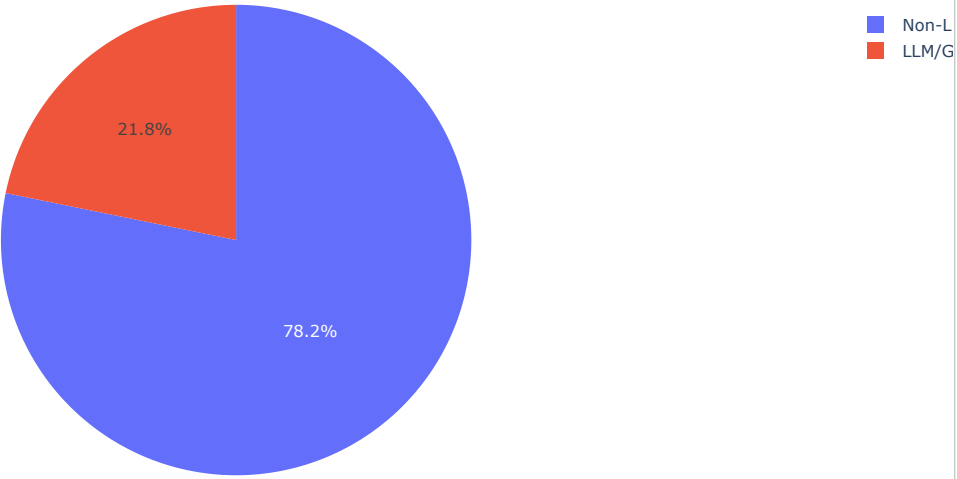
	count
is_llm_role	
0	1173
1	327

dtype: int64

```
fig = px.pie(
    values=llm_jobs.values,
    names=["Non-LLM", "LLM/GenAI"],
    title="LLM/GenAI Job Market"
)

fig.show()
```

LLM/GenAI Job Market



LLM Salary Premium

```

llm_salary = (
    df.groupby("is_llm_role")["annual_salary_usd"]
    .mean()
)

llm_salary

```

```

          annual_salary_usd
is_llm_role
0          191308.610401
1          207746.177370

dtype: float64

```

```

fig = px.bar(
    x=["Non-LLM", "LLM/GenAI"],
    y=llm_salary.values,
    title="LLM vs Non-LLM Average Salary",
    labels={
        "x": "Role Type",
        "y": "Average Salary (USD)"
    }
)

fig.show()

```



Salary Prediction Model Select Features

```

features = [
    "job_title",
    "job_category",
    "experience_level",
    "years_of_experience",
    "education_required",
    "country",
    "remote_work",
    "company_size",
    "industry",
    "demand_score",
    "is_llm_role"
]

target = "annual_salary_usd"

```

Remove missing values

```

model_df = df[features + [target]].dropna()

print(model_df.shape)

(1500, 12)

```

X and y

```
X = model_df[features]

y = model_df[target]
```

Train/Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training:", X_train.shape)
print("Testing:", X_test.shape)

Training: (1200, 11)
Testing: (300, 11)
```

Categorical columns

```
categorical_features = [
    "job_title",
    "job_category",
    "experience_level",
    "education_required",
    "country",
    "remote_work",
    "company_size",
    "industry"
]

numeric_features = [
    "years_of_experience",
    "demand_score",
    "is_llm_role"
]
```

Preprocessing

```
preprocessor = ColumnTransformer(
    transformers=[
        (
            "cat",
            OneHotEncoder(
                handle_unknown="ignore"
            ),
            categorical_features
        )
    ],
    remainder="passthrough"
)
```

Random Forest Model

```
rf_model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        (
            "model",
            RandomForestRegressor(
                n_estimators=300,
                random_state=42,
                n_jobs=-1
            )
        )
    ]
)
```

Prediction

```
rf_model.fit(X_train, y_train)
y_pred = rf_model.predict(X_test)
```

Model Evaluation

```
mae = mean_absolute_error(y_test, y_pred)

rmse = np.sqrt(
    mean_squared_error(y_test, y_pred)
)

r2 = r2_score(y_test, y_pred)

print("MAE:", mae)
print("RMSE:", rmse)
print("R² Score:", r2)
```

MAE: 18829.289444444446
RMSE: 25020.523415675823
R² Score: 0.847272582760384

Actual vs Predicted Salary

```
comparison = pd.DataFrame({
    "Actual Salary": y_test.values,
    "Predicted Salary": y_pred
})

comparison.head(20)
```

	Actual Salary	Predicted Salary	
0	233000.0	226923.333333	
1	184000.0	167980.000000	
2	218000.0	195693.333333	
3	244000.0	227096.666667	
4	191000.0	164782.000000	
5	336000.0	327784.666667	
6	240000.0	228726.666667	
7	224000.0	249676.666667	
8	145000.0	147173.333333	
9	145000.0	145486.666667	
10	140000.0	158880.000000	
11	348000.0	342320.000000	
12	239000.0	215700.000000	
13	206000.0	230149.333333	
14	126000.0	122040.000000	
15	157000.0	156816.666667	
16	250000.0	273346.666667	
17	240000.0	235522.666667	
18	157000.0	158150.000000	
19	257000.0	265536.666667	

Next steps: [Generate code with comparison](#) [New interactive sheet](#)

Visualization

```
fig = px.scatter(
    comparison,
    x="Actual Salary",
    y="Predicted Salary",
    title="Actual vs Predicted Salary",
    trendline="ols"
)

fig.show()
```

Actual vs Predicted Salary



Salary Prediction Function

```
def predict_salary(
    job_title,
    job_category,
    experience_level,
    years_of_experience,
    education_required,
    country,
    remote_work,
    company_size,
    industry,
    demand_score,
    is_llm_role
):

    input_data = pd.DataFrame({
        "job_title": [job_title],
        "job_category": [job_category],
        "experience_level": [experience_level],
        "years_of_experience": [years_of_experience],
        "education_required": [education_required],
        "country": [country],
        "remote_work": [remote_work],
        "company_size": [company_size],
        "industry": [industry],
        "demand_score": [demand_score],
        "is_llm_role": [is_llm_role]
    })

    prediction = rf_model.predict(input_data)[0]

    return round(prediction, 2)
```

```
predict_salary(
    "Data Scientist",
    "Data Science",
    "Mid",
    3,
    "Bachelor",
    "United States",
    "Fully Remote",
    "Enterprise",
    "Technology",
    85,
    0
)
```

```
np.float64(162003.33)
```

Career Intelligence

```
career_insights = (
    df.groupby("job_title")
```

```
.agg(
    average_salary=("annual_salary_usd", "mean"),
    average_demand=("demand_score", "mean"),
    demand_growth=("demand_growth_yoy_pct", "mean"),
    job_count=("job_id", "count")
)
.reset_index()
)

career_insights.head()
```

	job_title	average_salary	average_demand	demand_growth	job_count	
0	AI Agent Developer	225649.122807	96.0	49.896491	57	
1	AI Business Analyst	134145.161290	78.0	17.116129	62	
2	AI Compliance Manager	155939.393939	68.0	16.798485	66	
3	AI Engineer	159859.375000	95.0	52.750000	64	
4	AI Ethics Officer	148482.142857	72.0	17.058929	56	



Next steps: [Generate code with career_insights](#) [New interactive sheet](#)

Best career opportunities

```
career_insights["career_score"] = (
    career_insights["average_demand"] * 0.4 +
    career_insights["demand_growth"] * 0.3 +
    (career_insights["average_salary"] /
     career_insights["average_salary"].max()) * 100 * 0.3
)
```

```
career_insights.sort_values(
    "career_score",
    ascending=False
).head(10)
```

	job_title	average_salary	average_demand	demand_growth	job_count	career_score	
15	LLM Engineer	240960.000000	98.0	51.784000	75	83.469155	
24	Senior ML Engineer	247953.125000	96.0	49.017187	64	82.673026	
0	AI Agent Developer	225649.122807	96.0	49.896491	57	80.277114	
21	RAG Engineer	218226.415094	94.0	55.324528	53	80.220383	
16	ML Engineer	202581.818182	97.0	52.645455	55	78.751077	
19	NLP Engineer	210854.545455	91.0	54.694545	55	77.952309	
14	Generative AI Engineer	206760.563380	95.0	48.198592	71	77.115324	
17	MLOps Engineer	199215.686275	93.0	52.819608	51	76.801919	
3	AI Engineer	159859.375000	95.0	52.750000	64	72.887882	
12	Data Scientist	173639.344262	92.0	46.821311	61	71.552507	



Interactive Dashboard in Colab

⌵ KPI cards

```
total_jobs = len(df)

avg_salary = df["annual_salary_usd"].mean()

median_salary = df["annual_salary_usd"].median()

avg_demand = df["demand_score"].mean()

print("Total Jobs:", total_jobs)
print("Average Salary:", round(avg_salary, 2))
print("Median Salary:", round(median_salary, 2))
print("Average Demand:", round(avg_demand, 2))
```

Total Jobs: 1500
Average Salary: 194892.0
Median Salary: 180000.0
Average Demand: 87.52

Interactive Salary Chart

